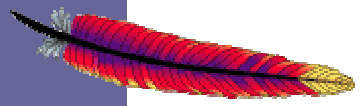
Jasper, the JSP compiler in Tomcat 5

Kin-man Chung

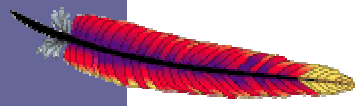
Sun Microsystems, Inc.

ApacheCon 2003
11-17-2003, Session MO10, 14:30-15:30



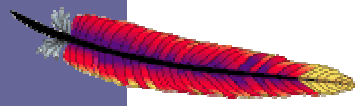
Overview

- What is Jasper?
- Jasper architecture
- JSP 2.0 implementation
- Optimizations and improvements
- Q & A



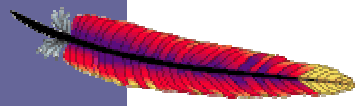
What is Jasper?

- Component of Tomcat that compiles a JSP page into servlet
  - .jsp .jspx .tag .tagx to .java .class
- New in Tomcat 5, aka Jasper 2
- Implements JSP 2.0



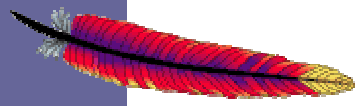
Jasper Design

- Compiler organized into modules
- Each module has simple function and task
- Multiple phase or pass
- Each pass driven by internal representation of the compilation unit
- Some passes are optional



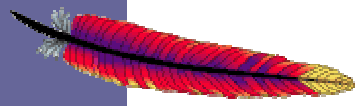
Compilation Steps

- Parser
- Validator
- Collector
- TagFileProcessor
- TagPluginManager
- Generator
- SMAP Generator



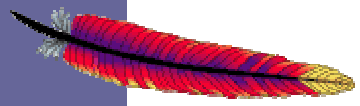
Parser

- XML syntax
  - Uses SAX parser
- Standard syntax
  - Context sensitive
  - Recursive descent
  - Based on EBNF grammar



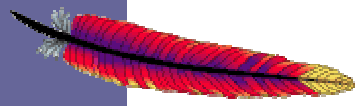
Internal Representation

- Node
  - Content for current compilation unit
- PageInfo
  - Compilation unit information
- TldLocationsCache
  - Taglib uri-to-tld mapping
- TagLibraryInfoImpl
  - TagLibraryInfo for custom tags



o.a.jasper.compiler.Node

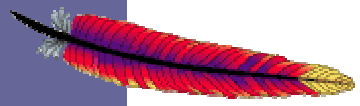
- Base class
  - Attributes, text, body, source location
- Extended classes
  - Represent various syntax elements
    - Directive, scriptlet, EL, standard/custom action
- Node$Nodes
  - Represents a page or body
- Node$Visitor
  - For nodes traversal



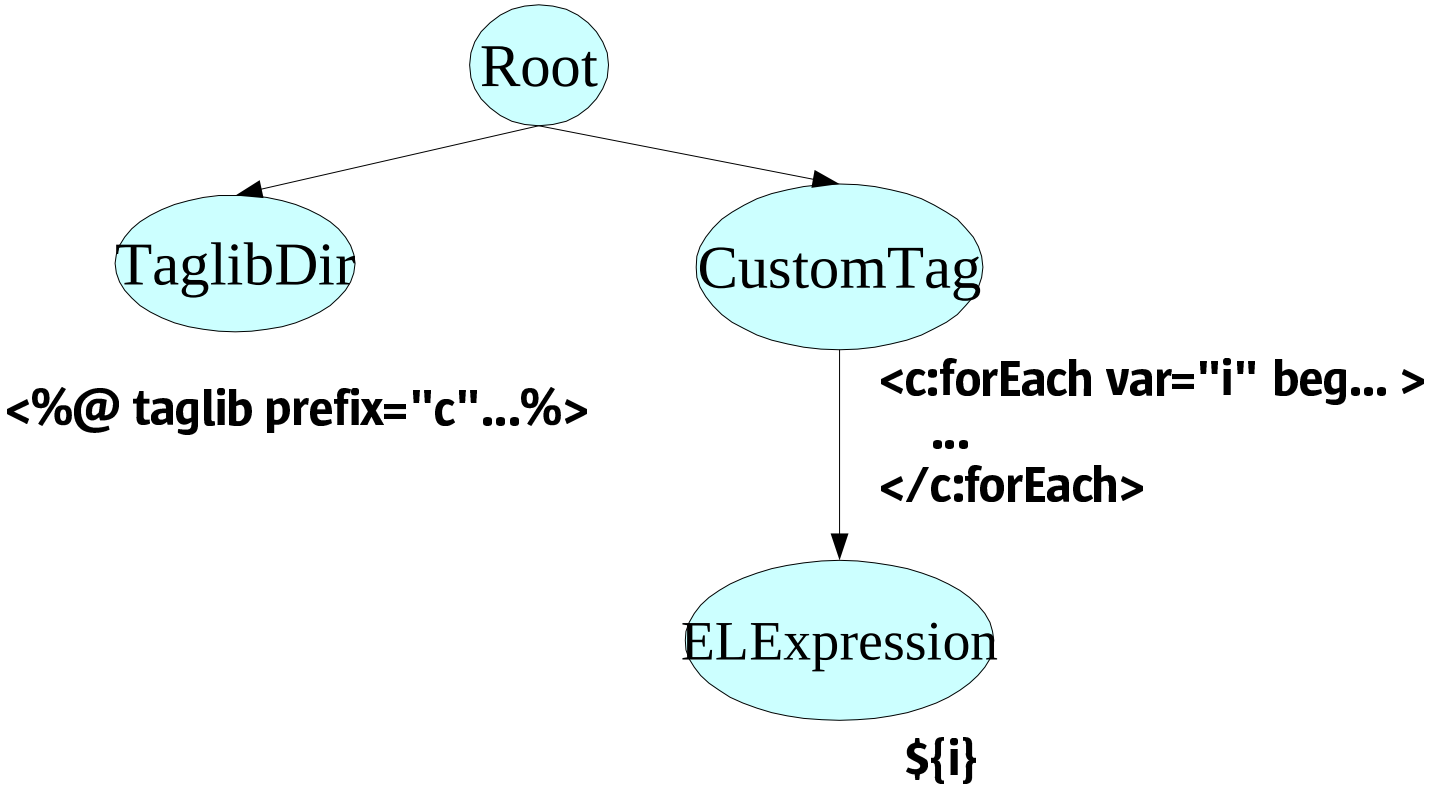
Example: iter.jsp

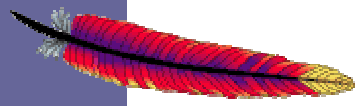
```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>

<c:forEach var="i" begin="1" end="10">
        ${i}
</c:forEach>
```



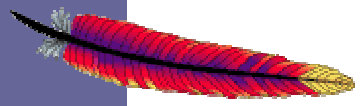
Internal Representation of iter.jsp





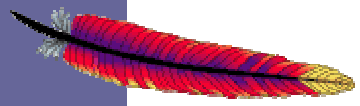
Validator

- Validates attributes in page elements
  - Checks for mandatory and optional attributes
  - Checks for validity and consistency
- Special custom tag attribute check
- Validates XML view
- Processes attributes that are expressions
- Processes page and tag directives
  - Stores in PageInfo for use in other steps



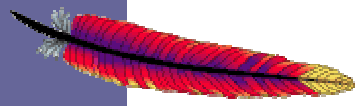
Collector

- Collects information about page
  - Tag nesting level
  - Is a body scriptlet free?
- Information will be used to generate better codes



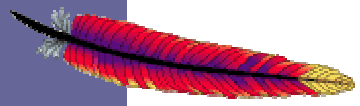
TagFileProcessor

- Parses tag files for directives only
  - Equivalent to parsing a tld entry
- Compiles tag files referenced and used
  - Generates tag handler .class files



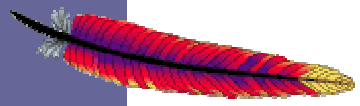
Tag Plugins

- Tag plugin
  - Generates java codes instead of tag handler calls
- Jasper provides a framework for writing tag plugins
- Currently provides some Plugins for some JSTL tags



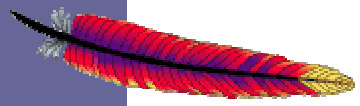
TagPluginManager

- Manages tag plugins
  - Determines when plugins can be used
  - Loads plugins
  - Invokes plugins during code generation
- Provide an implementation for TagPluginContext that is used by Plugins



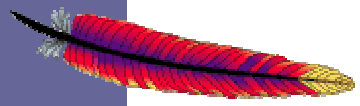
Generator

- Generates .java file from Nodes
- Implements a visitor for each element type
- Shared components for JSP and tag file
- Multiple pass
- Allows out-of-order code generation with the use of output buffers



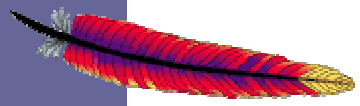
SMAP Generator

- Supports JSR-45 for JSP debugging
- Generates SMAP during compilation
- Incorporates SMAP into class file after javac compilation
- SMAP generation can be suppressed
- SMAP can be dumped



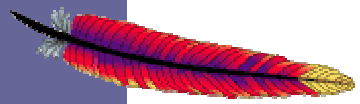
JSP 2.0 Implementation

- Expression Language
- JSP Fragments
- SimpleTag Handlers
- Tag Files
- JSP Configuration
- Page Character Encoding



Expression Language (EL)

- Introduced in JavaServer Pages Standard Tag Library (JSTL)
- Can be used instead of scriptlet expressions
- An implementation in Jakarta-commons



EL Functions: Example

/WEB-INF/functions.tld:

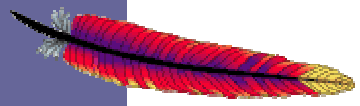
```
<taglib xmlns="http://java.sun.com/xml/ns/j2ee" version="2.0">
    ....
  <function>
    <description>Reverses the characters in the given String
    </description>
    <name>reverse</name>
    <function-class>jsp2.el.Functions</function-class>
    <function-signature>java.lang.String reverse(java.lang.String)
    </function-signature>
  </function>
</taglib>
```

```
<%-- example.jsp --%>
<%@ taglib prefix="my" uri="/WEB-INF/functions.tld"%>
${my:reverse("abcdef")}
```



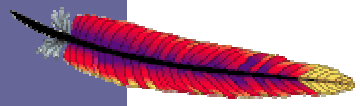
EL Implementation

- EL Recognition in parser
  - Template text and Attribute
- ELParser: parsing EL
  - Currently only parses functions
- Generate codes to invoke ExpressionEvaluator in common-el
- To-do: full ELParser



EL Function Implementation

- ExpressionEvaluator uses a FunctionMapper to map prefix to tag class
- Cannot use a static global FunctionMapper
- Jasper generates a FunctionMapper for each function



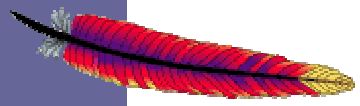
EL Function Implementation

```
/* example_jsp.java */

static _jspx_fnmap_0 =
    ProtectedFunctionMapper.getMapForFunction(
        "my:reverse", jsp2.examples.el.Functions.class, "reverse",
        new Class[] {java.lang.String.class});

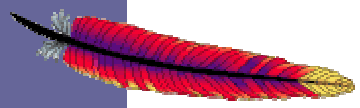
public void _jspService() {

    out.write(PageContextImpl.proprietaryEvaluate(
        "${my:reverse(\"abcde\")}", java.lang.String.class,
        pageContext, _jspx_fnmap_0));
}
```



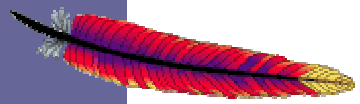
JSP Fragments

- Encapsulates JSP code, which can be
  - Body of named attribute (<jsp:attribute>) declared to be a fragment
  - Body of any tag handled by a Simple Tag handler
- Passed to tag handler as instance of javax.servlet.jsp.tagext.JspFragment
- Tag handler executes fragment body by calling fragment's invoke() method



SimpleTag Handler

- Provides new, much simpler tag extension API
- Invoked via doTag() (instead of doStartTag(), doEndTag())
- Container encapsulates tag body in JSP fragment and passes it to handler via setJspBody()
- Tag handler evaluates body by calling getJspBody().invoke()



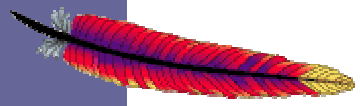
SimpleTag Example

```
<%-- example.jsp --%>
<%@ taglib prefix="my" tagdir="/WEB-INF/tags" %>

<my:repeat num="4">

    Invocation #${index}: Hello, world!  <-- JSP Fragment

</my:repeat>
```



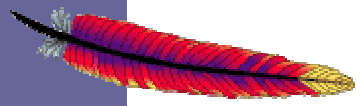
Tag File Example

```
<%-- repeat.tag --%>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<%@ attribute name="num" %>
<%@ variable name-given="index" scope="NESTED" %>

<br>
<c:forEach var="index" begin="1" end="${num}">

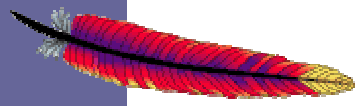
    <jsp:doBody/><br>

</c:forEach>
```



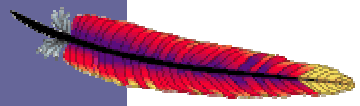
SimpleTag Example: Output

Invocation #1: Hello, world!
Invocation #2: Hello, world!
Invocation #3: Hello, world!
Invocation #4: Hello, world!



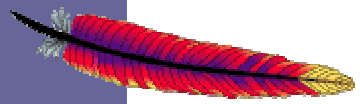
SimpleTag Example: codes

```
/* example_jsp.java */
// Instantiate and initialize tag handler
_jspx_th_my_repeat_0 = new org.apache.jsp.tagfile.repeat();
_jspx_th_my_repeat_0.setJspContext(pageContext);
// Call setter method for XML-style attribute “num”
_jspx_th_my_repeat_0.setNum("4");
// Encapsulate tag body in JSP Fragment
_jspx_th_my_repeat_0.setJspBody(
        new example_jspHelper(
          0, pageContext, _jspx_th_my_repeat_0));
// Invoke tag
_jspx_th_my_repeat_0.doTag();
```



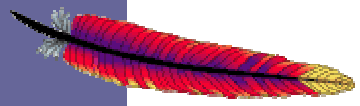
JSP Fragment Implementation

- Helper class avoids generating separate class for each fragment
- Helper class implements java.servlet.jsp.tagext.JspFragment
- One helper class per page
- Helper class constructor accepts discriminator, which selects which fragment the generated instance will execute



Tag File Processing

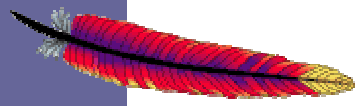
- JSP Page Parsing Phase
  - Special parse mode for directive info only
- JSP Page Compilation Phase
  - Suspenses current compilation
  - Compiles tag file and load tag handler
  - Special handling for recursive tag file invocation



JSP Configuration

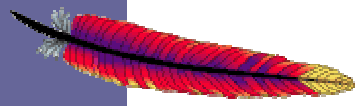
- Configuration info rooted on <jsp-config> element in web.xml
- <jsp-property-group> subelement lists all properties applicable to JSP pages that match URL pattern:

```
<jsp-property-group>
 <url-pattern>*.jsp</url-pattern>
 <el-ignored>true</el-ignored>
</jsp-property-group>
```



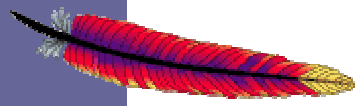
JSP Configuration Implementation

- Parses web.xml and cache JSP configuration information
- Before compilation, determines which properties (if any) apply to current translation unit, and store them in PageInfo
- Only for top level pages



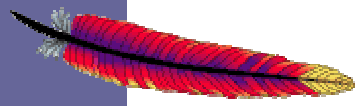
Page Character Encoding

- Determined for each file separately
- Can be specified in JSP configuration
- Encoding for JSP documents (XML syntax) is determined in accordance with XML spec
- Jasper implements autodetected XML encoding since encoding cannot be obtained from SAX parser



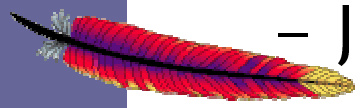
Optimizations in Jasper

- Removal of try/catch/finally blocks
- Tag pooling/reuse
- Buffer reuse in implementation of PageContext.pushBody()
- Separation of custom tag invocations into their own methods (to address 64K limitation)
- Tag PlugIns



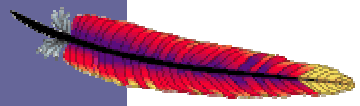
Other Improvements

- Add source dependencies for
  - Pages referenced via include directives
  - Tag Library Descriptor (TLD)
  - Tag files
- JSPC
  - More shared codes with embedded compilation
  - Can be configured as an ant task
- Global taglibs that can be shared among web applications
  - Just drop jar in tomcat/shared/lib



What's Next

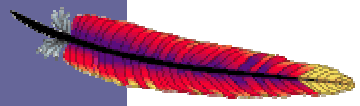
- Currently JSP 2.0 feature complete
- Improve quality
- Improve performance
- Third party IDE/tools support
- Suggestions and involvements welcome



Q&A

Kin-man Chung
Sun Microsystems, Inc.

kchung@sun.com



This presenation has ended.
Click anywhere to return.

_

